

# Deltazium — CDC 파이프라인 구축·운영 기록

## Debezium + Kafka 기반 변경 데이터 캡처, changelog 보관·복구(replay) 아키텍처, 웹 제어 콘솔

최남희 · 단독 설계·개발 · 2026.07 - 08 · 상용 CDC 솔루션 개발 경험 기반의 검증 프로젝트

## 1. 개요

소스 DB의 변경(CDC)을 Debezium으로 캡처해 같은 스트림을 두 갈래로 병행 적재한다(검증 시점의 소스·타깃은 Oracle, 캡처는 LogMiner). 타깃 DB에는 현재 상태를(실적재), Iceberg/S3에는 모든 변경 이력을(append-only changelog) 쌓는다. changelog는 미러링이 아니라 특정 시점부터의 복구(replay)를 위한 원본이며, 이 복구 아키텍처가 실제 배선에서 성립하는지 증명하는 것이 프로젝트의 목적이었다. 그 위에 테이블 등록·사전 점검·DDL 승인·복구를 담당하는 제어면(Spring Boot + React)을 얹었고, 운영 중 실제 발생한 장애들을 계기로 장애 감지 → 원인 표시 → 단계형 복구, 그리고 로그·상태를 스스로 조회해 원인을 짚는 AI 진단 어시스턴트까지 운영 사이클을 확장했다.

규모 — Java 9.9k · TypeScript 5.3k 라인, 자동화 테스트 136건, 실트래픽 CDC 이벤트 53만+, 실장애 4건 전건 원인 확정 후 복구 (유실 0).

## 2. 아키텍처

Oracle(SRC) —Debezium source(LogMiner)→ Kafka(KRaft) —┐ Debezium JDBC sink → Oracle(TGT) · 실적재  
└ Iceberg sink(append) → MinIO(S3) · changelog

복구: recovery-job —(Iceberg scan · SCN 순서 복원 · envelope 재조립)→ 복구 토픽 → recovery-sink → Oracle(TGT)

[Web UI(React)] ≠ [backend(Spring Boot): 등록·사전 점검 · DDL 승인 · 복구 · 이벤트 이력 · 실측 메트릭 · AI 진단]

| 컴포넌트            | 역할                                                   | 보존/특성                      |
|-----------------|------------------------------------------------------|----------------------------|
| Kafka (KRaft)   | 짧은 버퍼 · fan-out — 두 sink가 독립 consumer group으로 lag 격리 | retention 24h, 이력 보관 책임 없음 |
| Iceberg / MinIO | 장기 이력 · 복구 원본 (envelope-as-is, append-only)          | 1일 파티션 drop이 곧 보존 정책       |
| Oracle 타깃       | 현재 상태 (PK upsert 먹등 apply)                           | —                          |

데이터 경로에는 기성 커넥터만 쓰고 자체 코드는 제어면·복구·UI에만 둔다. apply 시맨틱을 JDBC sink 단일 경로로 유지해, 복구 결과가 라이브 적재와 달라지는 사고를 구조적으로 차단하기 위함이다.

### 3. 설계 판단

#### 3.1 멱등 apply를 전제로 한 복구 단순화

전달 보장은 at-least-once로 두고, apply를 PK 기반 upsert(MERGE)로 통일해 몇 번을 재적용해도 결과가 같게 했다. 복구 재생의 경계 중복을 정밀 절단 없이 흡수하는 근거이며, PK 없는 테이블은 등록 단계에서 거부한다. 복구는 재발행 단일 경로 — recovery-job은 Iceberg를 읽어 복구 토픽에 재발행까지만 하고, apply는 라이브와 동일한 JDBC sink 설정이 담당한다.

#### 3.2 changelog 스키마를 실측 실험으로 확정

원안(평탄화 스키마)이 기성 커넥터·SMT 조합으로 구현 불가함을 실험으로 증명하고 (스톡 SMT는 중첩 필드 승격 불가, 기존 변환 SMT는 before 이미지 소실 — 무손실 재조립 불변식 위반), envelope 구조 그대로 저장 + backend가 테이블 사전 생성 + 1일 파티션으로 설계를 개정했다. envelope 왕복 테스트(원본 → Iceberg 행 → 재조립 동등성)가 이 불변식의 회귀 방어선이다.

#### 3.3 캡처 장애의 재현·진단·전환 — 유실 0

공유 dev Oracle에서 LogMiner redo\_log\_catalog 전략이 ORA-1371(complete dictionary not found) 재시도 루프에 빠져 커넥터는 RUNNING인데 캡처가 멈추는 장애를 재현했다. 디셔너리 아카이브 존재를 확인해 전략 자체의 취약점으로 진단하고 online\_catalog로 전환(DDL 이벤트는 양 전략 모두 발행됨을 확인해 승인 워크플로 유지), 소스 redo 재마이닝으로 밀린 9,673건을 전량 회수했다. 커넥터 삭제 후 재등록 시 잔존 offset으로 스냅샷이 건너뛰어지는 함정도 실측으로 발견해 등록 해제 시 offset 정리를 자동화했다.

#### 3.4 캡처 계정 최소 권한 8개 도출

Debezium 문서의 보수적 권한 목록(20여 개)을 실 배선의 권한 실패를 재현하며 8개로 압축했다. SELECT ANY DICTIONARY가 V\$ 뷰 개별 grant를 대체함을 확인하고, DBMS\_LOGMNR\_D(디셔너리 전략 전용)·FLASHBACK ANY TABLE(초기 스냅샷 전용)의 용도를 구분해 문서화했다. 사전 점검이 권한 누락 시 DBA용 GRANT 스크립트를 생성해 안내하고 배포를 차단한다.

#### 3.5 운영 동작의 원칙화

jdbc-sink를 테이블별 커넥터로 분리해 타깃 이름 매핑과 테이블 단위 정지(DDL 거부 격리)를 가능하게 했고, recovery-sink는 apply 완료(consumer lag 소진)를 감지해 자동 정지한다(평시 정지 원칙). 복구 트리거의 자동 재개 옵션으로 "정지 → S3 복구 → 라이브 복귀(go-live)"가 한 번의 조작으로 완결된다 — 경계 중복은 전부 멱등이 흡수하므로 절단 시점 계산이 필요 없다.

#### 3.6 실장애: connector RUNNING / task FAILED — 나홀 미감지의 해부와 복구

공유 dev Oracle의 archive log 정리로 재개 지점 SCN이 소실돼 캡처 task가 죽었는데, connector는 RUNNING·발행이 없으니 lag도 0 — 모든 지표가 "평온"을 가리키는 나홀이 흘렀다. Connect REST의 task trace로 원인을 확인하고 이 장애에서 기능을 파생시켰다: task 포함 상태 판정(최악값 집계), 장애 배너(감지 시각·원인·전체 trace), 행별 재시도, 단계형 재스냅샷 워크플로. 최종 복구는 타깃 truncate 완전 재구축으로 10.5만 행을 50초에 go-live — 갭 중 소스에서 삭제된 행(고아 +2,237)까지 정리했다. "lag 0은 따라잡음과 무소식을 구분하지 못한다"는 교훈은 대시보드의 발행 vs apply 겹침 차트로 만들었다.

3.7 단계형 재스냅샷 — 승인·홀드가 있는 상태 기계

재스냅샷을 블랙박스 버튼이 아니라 관찰 가능한 단계로 만들었다: 유입 차단 → 파이프 잔량 소진(실시간 lag) → 타깃 비우기 승인(시스템 실행은 권한 사전 점검, 직접 실행은 DBA 홀드 — 테이블별 count=0 폴링으로 자동 진행) → 재기동 → 스냅샷 → go-live. 되돌릴 수 없는 지점(offset 삭제) 전까지는 취소·실패 시 원복이 보장되고, 스냅샷 진행률은 Debezium notification 채널을 소비해 추정이 아닌 실측으로 표시한다.

3.8 모니터링: 자체 경량 수집 → 2단 롤업

1분 샘플러(토픽 offset 델타·sink lag·pid 기반 /proc CPU·RSS)를 PostgreSQL에 적재하고, 완결 구간만 대상으로 분(48h) → 시간(60일) → 일(1년) 롤업한다. NOT EXISTS 먹등이라 재실행 중복이 없고, 집계는 지표 성격대로 처리량은 합, lag은 구간 최대로 스파이크를 보존한다. Prometheus/Grafana는 의도적으로 미도입(백로그) — 무엇을 보여줘야 하는가를 먼저 검증했다.

3.9 재개 지점(offset SCN) 유실 — 같은 예외, 세 경로의 방어

같은 종착점(LogFileNotFoundException)의 장애가 서로 다른 세 경로로 실발생했다: 커넥터 다운 중 archive 정리, 유틸 트래픽 중 offset 정체(Debezium offset은 이벤트 발행 시에만 영속화되므로 "유틸 기간 > 보존 기간"이면 결정론적으로 재현), 미커밋 장기 트랜잭션의 시작 SCN pin(heartbeat로도 전진 불가 — 후속 트랜잭션 처리는 막히지 않고 재시작 지점만 잡힌다). 경로별 방어를 공식 문서 검증 후 적용했다: heartbeat.interval.ms 60s(유틸) · archive 보존 기간(다운타임) · log.mining.transaction.retention.ms 2h(장기 트랜잭션 — 버려진 트랜잭션이 이후 커밋되면 복제에서 빠지는 대가를 문서화하고 선택). 롤오버 index가 시각 순서와 무관한 회전 슬롯이라는 함정 등 진단 과정의 발견도 장애 기록으로 남겼다.

3.10 AI 진단 어시스턴트 — tool use 기반 임베디드 에이전트

"왜 CDC가 멈췄어?" 질문을 받으면 Claude가 읽기 전용 도구 2종(파이프라인 상태 요약 · 로그 검색)을 스스로 골라 호출해 원인을 짚는다 — 실시간 상태 진단이라 RAG가 아닌 tool use를 선택했다. 도구 표면은 AI용으로 별도 설계(비정상만 상세 · 서버측 응답 캡 · 근거 위치 포함)하고 같은 프로세스에서 직접 실행하며, 조치 실행 도구는 의도적으로 두지 않았다 — 제안까지만, 실행은 사람이 화면에서 한다. 모든 주장에 로그 파일·줄 번호 근거를 강제해 답변을 grep으로 검증할 수 있다. tool use 왕복의 반복 전송을 prompt caching으로 흡수해 진단 1건 비용을 \$1 → \$0.06으로 낮췄다(실측 — 왕복별 토큰 로깅으로 캐시 적중 회귀 감시).

4. 웹 콘솔 기능

| 화면         | 기능                                                                                                                                                          |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 대시보드       | KPI 카드(커넥터·처리량·최대 lag·미승인 DDL) + 토폴로지(자체 SVG — 교차 없는 직교 라우팅·흐름 애니메이션) + 주의 필요 목록(lag 초과·DDL 대기, 해당 화면 이동) + 처리량(발행 vs apply)·lag 시계열(테이블·기간 선택) + 최근 운영 이벤트 |
| CDC 등록 위저드 | 6단계: 소스 선택 → 디렉터리 조회(와일드카드 전개, PK 없는 테이블 선택 차단) → 타깃 → 컬럼 매핑(제외·변환식·구문 검증) → 사전 점검(supp.log 승인 후 적용 · 권한 GRANT 안내) → 스냅샷 모드 선택·배포                           |
| 테이블 모니터링   | 상태 필터 칩 + 그리드(실측 이벤트·lag·15분 추세), 행 선택 시 상세 drawer(상태·lag 차트·이벤트 이력·정지/재개/재시도/재스냅샷/삭제), 캡처 장애 배너(원인·trace·복구 진입)                                            |

|          |                                                                                                                    |
|----------|--------------------------------------------------------------------------------------------------------------------|
| DDL 이력   | schema change topic 상시 수집(retention 만료 대비 DB 적재), 승인(타깃 이름 치환 실행)/거부(테이블 단위 apply 정지), 비전파성 DDL 자동 무시              |
| 이벤트      | 테이블별 운영 이력 — 등록·정지-DDL·복구·커넥터 장애 전이(원인 trace 포함) 타임라인                                                              |
| 복구       | changelog(S3) 현황(용량·보존 구간·파티션별)·실행 이력 + 단계형 실행 drawer(대상 테이블 → SCN 범위 → 정합 검증)·go-live 자동 재개·SRC/TGT 정합 검증(행수+체크섬) |
| DB 연결    | 소스/타깃 연결 카드 — 접속 정보·사용 중인 테이블 수, 인라인 연결 테스트(카드 액센트로 결과 표시)                                                         |
| AI 진단 위젯 | 전 화면 공통 플로팅 채팅 — 상태 요약·로그 검색 도구 자율 호출, 로그 라인 근거와 함께 원인 제시(SSE 스트리밍·사용 도구 표시), 조치는 제안만                              |

## 5. 검증 결과

| 항목                  | 내용                                                                                                     |
|---------------------|--------------------------------------------------------------------------------------------------------|
| 실 Oracle end-to-end | 초기 스냅샷(30,088행) → LogMiner 실시간 스트리밍 → 타깃 apply(lag 0) + changelog 커밋까지 실패픽으로 확인                        |
| 캡처 장애 전환            | ORA-1371 재현 → online_catalog 전환 → 밀린 9,673건 유실 0 회수                                                    |
| 자동화 테스트 136건        | envelope 왕복 동등성·changelog 사전 생성(실 카탈로그)·복구 재발행 스모크(73건)·로그 검색·상태 요약·chat 단위 테스트 포함                     |
| 사전 점검 게이트           | LogMiner 권한 누락 감지 → GRANT 안내 → 부여 후 통과 (실동작)                                                           |
| 실장애 복구              | archive log 소실로 죽은 캡처(나홀 경과)를 재스냅샷 워크플로로 복구 — truncate 완전 재구축 10.5만 행·50초 go-live, 진행률 notification 실측 |
| 정합 검증               | 행 수 + ORA_HASH 체크섬으로 고아 행 편차(+2,237) 검출 → 재구축 후 수렴 확인                                                  |
| SCN 유실 방어 실증        | heartbeat 적용 후 heartbeat 토픽 1,459건/일 발행 확인, 세 번째 경로(장기 트랜잭션 pin) 재발을 로그로 원인 확정 후 retention 적용          |
| AI 진단 실전            | 재발 장애(offset SCN 유실)를 위젯 질문 한 번으로 원인 확정 — connect.log 라인 근거 인용, 실패 1시간 전 선행 경고(장기 트랜잭션)까지 로그에서 발굴      |
| 모니터링 실측             | 분당 2,262건 버스트에서 발행·apply 추적, lag 스파이크·소진 시계열 기록, 롤업 먹등 테스트                                             |

## 6. 기술 스택 · 문서화

Java 21 · Spring Boot 3.5 · MyBatis · Debezium 3.6 (Oracle LogMiner · JDBC sink) · Kafka 4.3 KRaft · Apache Iceberg 1.11 · MinIO(S3) · PostgreSQL 15 · React 19 · TypeScript · Vite · Tailwind 4 · shadcn/ui · TanStack Table · 자체 SVG(토폴로지·차트) · Claude API(tool use · SSE).

설계 기준(architecture.md) · 운영 가이드(operations.md) · 구현 내부 노트(internals.md) · 실장애 기록(incidents/, 건별 타임라인·진단 경로·교훈) · 실측 실험 기록(experiments/)을 리포에 유지 — "UI에는 사용자 결정에 필요한 것만, 시스템 내부는 문서·주석에" 원칙. 남은 과제: 기동 중 테이블 추가 시 초기적재(incremental snapshot + Kafka signal) · Prometheus/Grafana(JMX exporter) 도

입 후 자체 수집과 비교 · 컬럼 리네임의 적재 반영(스톡 sink 한계 — 정책 결정 대기) · S3 복구 폴 리허설 실행 검증 · AI 진단 멀티턴  
(후속 질문 맥락 유지) · offset 정체 조기 경보(실패 전 WARN의 운영 이벤트 승격)

## 부록 A. 콘솔 화면

개발 환경 화면 캡처이며, 내부 주소 등 일부 정보는 가림 처리했다.

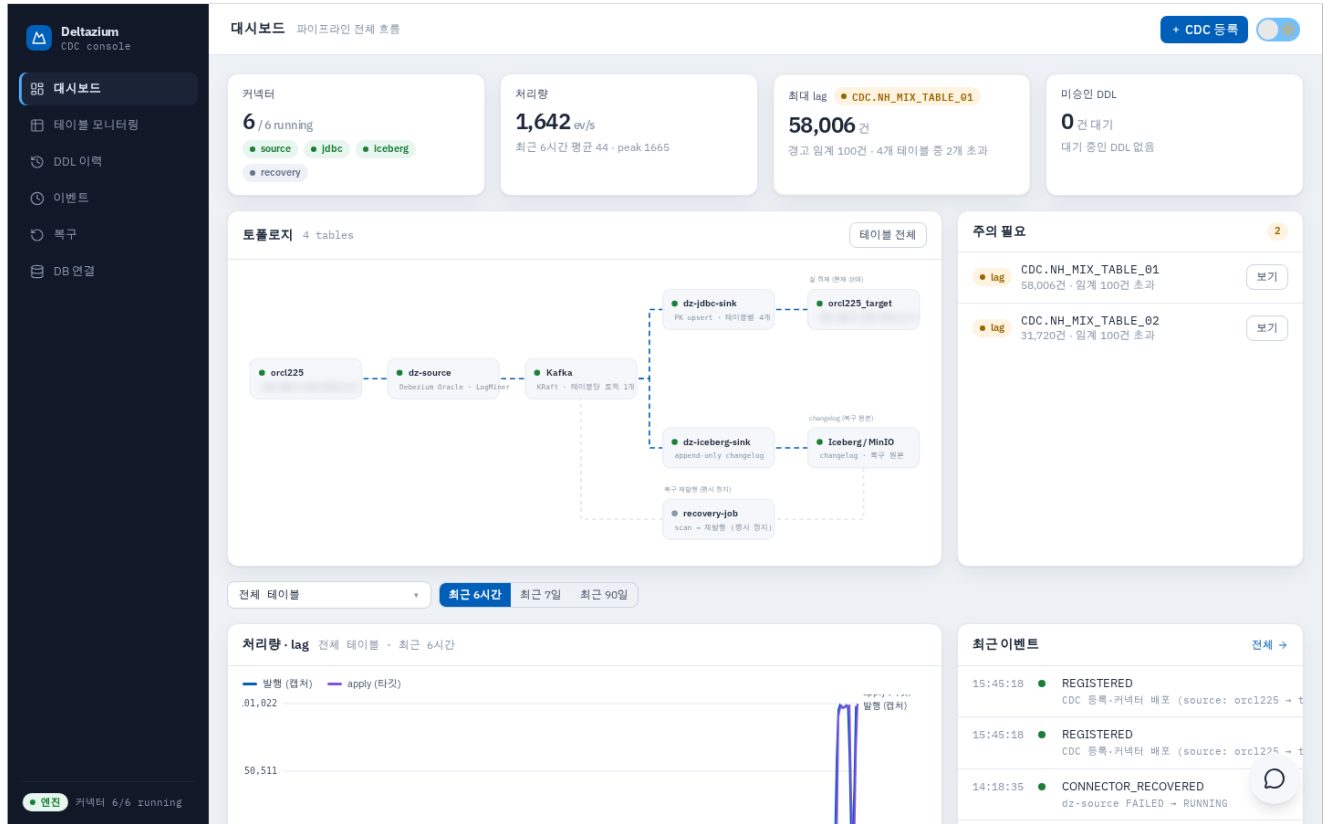


그림 1. 대시보드 — KPI 카드(커넥터·처리량·최대 lag·미승인 DDL)·토폴로지(자체 SVG)·주의 필요·처리량(발행 vs apply)·lag 시계열 (약 1,600 ev/s 버스트 실측)

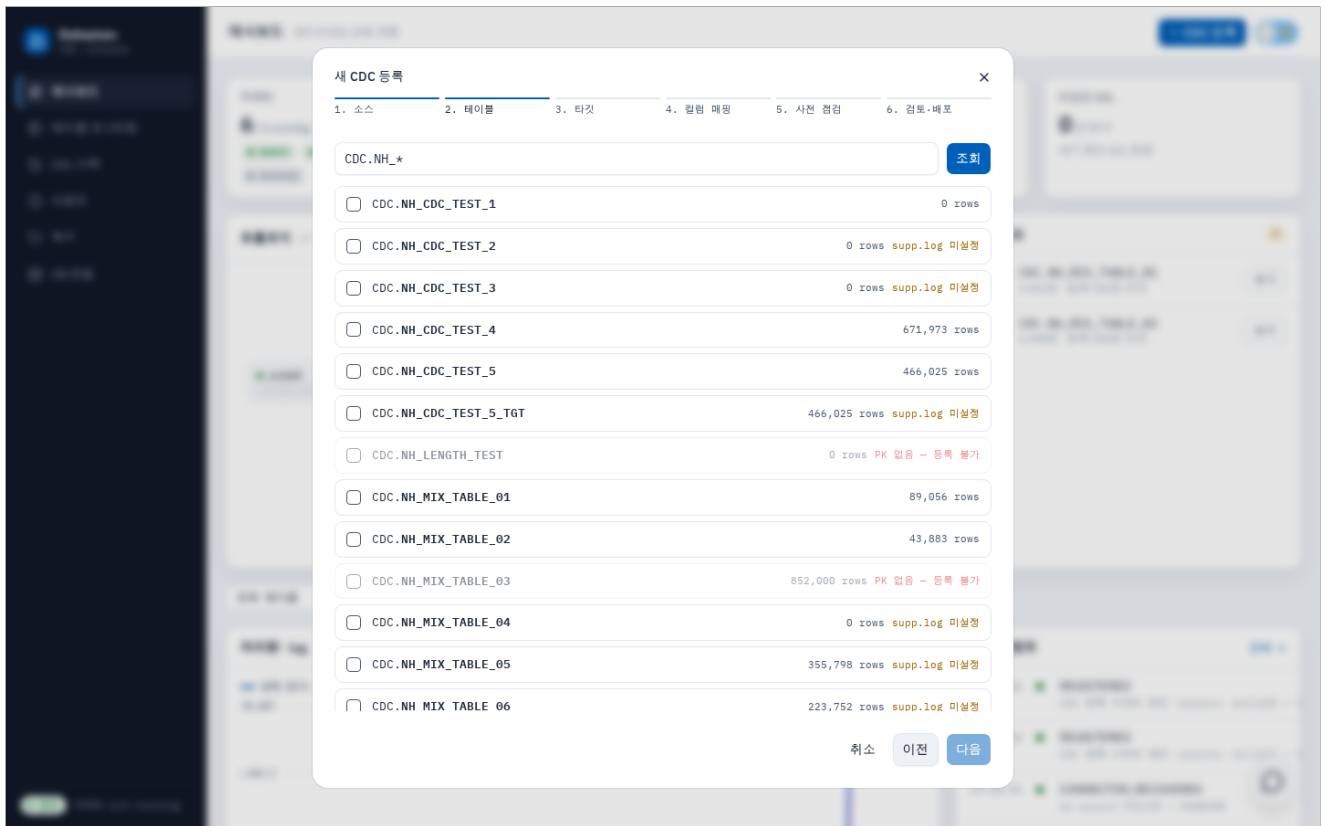


그림 2. 등록 위저드 — 소스 디렉터리 조회: PK 없는 테이블 선택 차단·supplemental logging 경고·행수

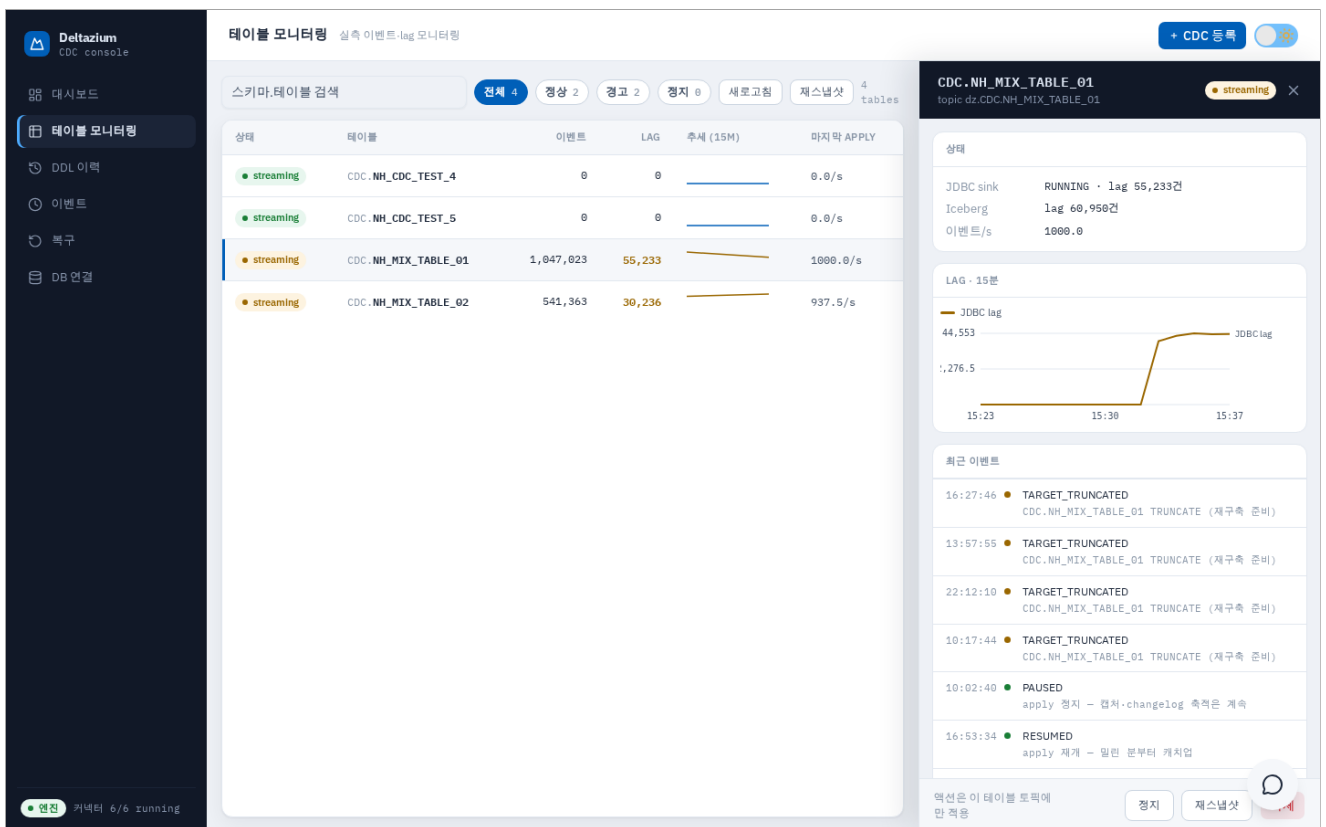


그림 3. 테이블 모니터링 — 실측 이벤트 수·이벤트/s·sink별 lag, 정지/재개/삭제

그림 4. 복구 — changelog(S3) 현황(보존 구간·파티션)·SCN 지정 재발행·go-live 자동 재개·정합 검증

그림 5. 운영 이벤트 타임라인 — 등록·정지·DDL·복구·커넥터 장애 전이(원인 trace 펼침)



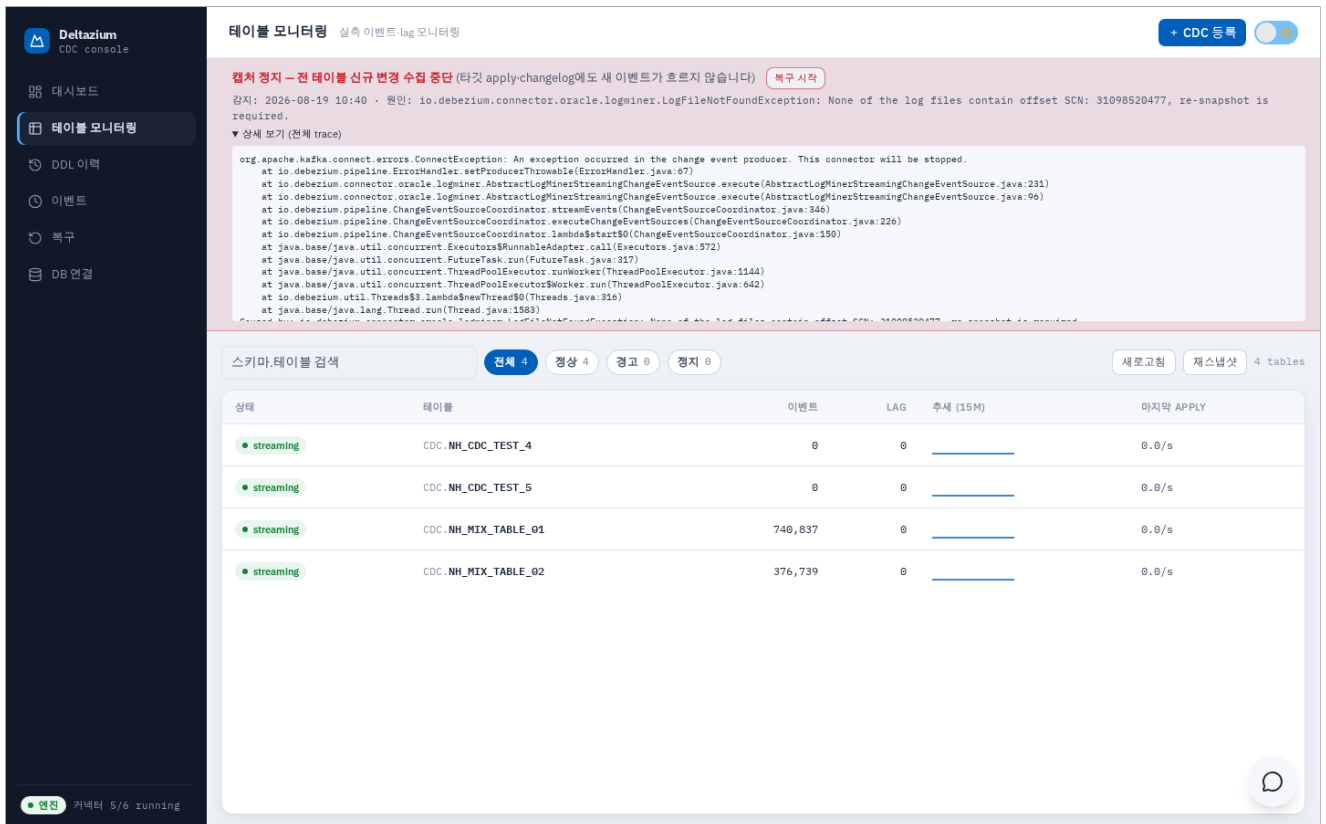


그림 6. 캡처 장애 경고 배너(실장에 실측) — connector RUNNING/task FAILED 불일치, 감지 시각·원인·전체 trace·복구 진입

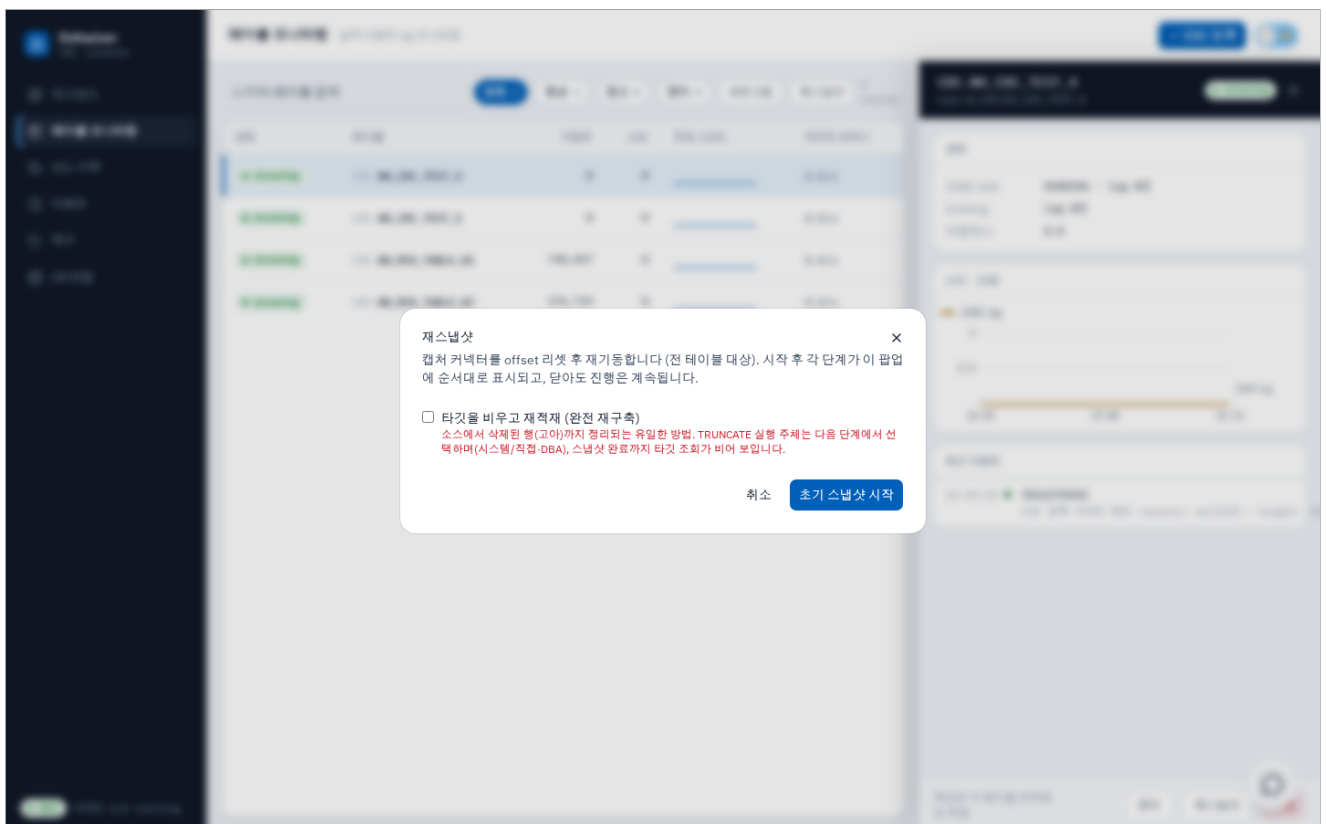


그림 7. 단계형 재스냅샷 — 시작 전 설정(타깃 비우기 여부·실행 주체). 시작 후 유입 차단→잔량 소진→타깃 비우기→재가동→스냅샷→go-live 단계가 같은 팝업에 순서대로 표시(실장에 복구 run: 10.5만 행·50초)

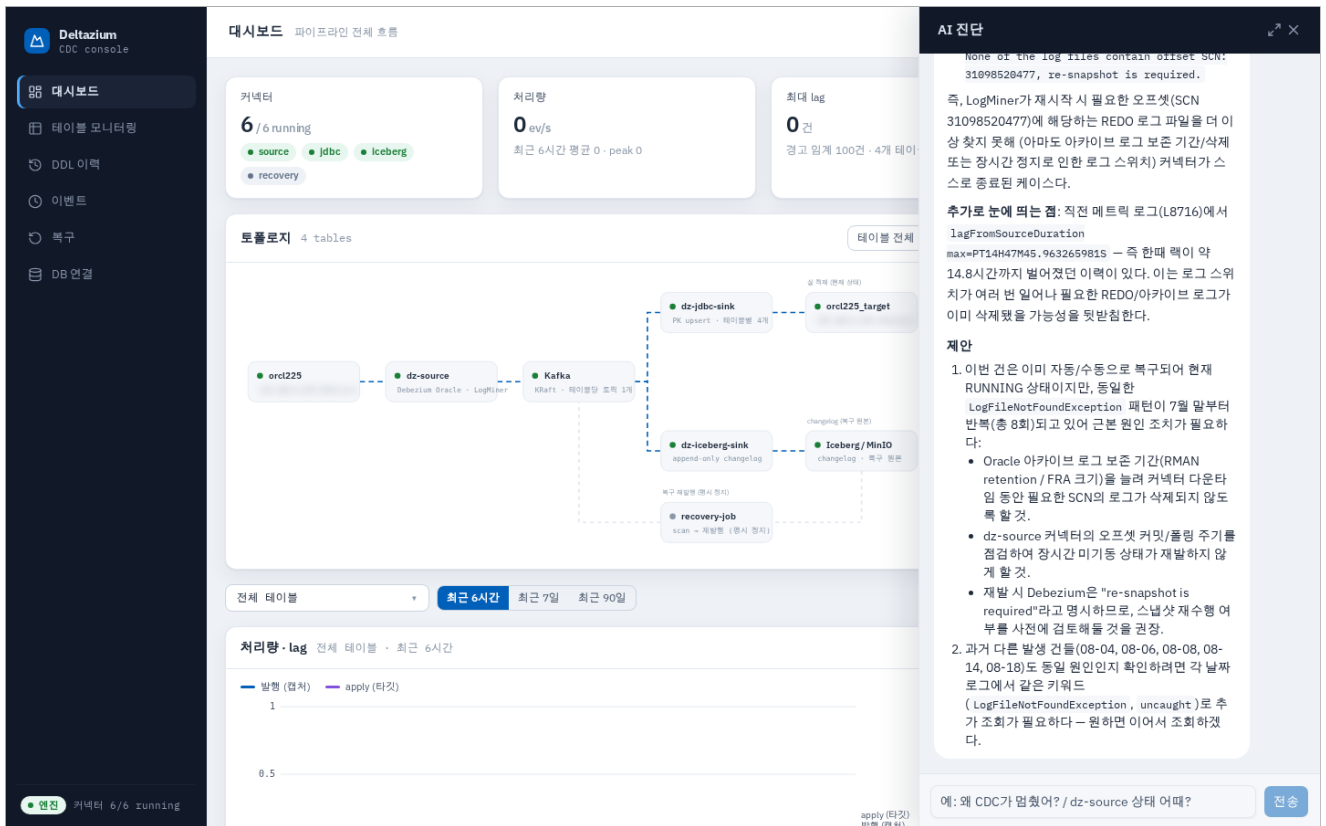


그림 8. AI 진단 위젯 — "dz-source 상태 어때? 최근 장애 있었어?"에 도구 자율 호출로 응답: 8월 반복 장애(8회)를 아카이브 로그 보존 기간 < offset 커밋 지연으로 진단, 같은 시점 테이블 lag 상관관계 확인·조치 3건 제안(실측 대하)